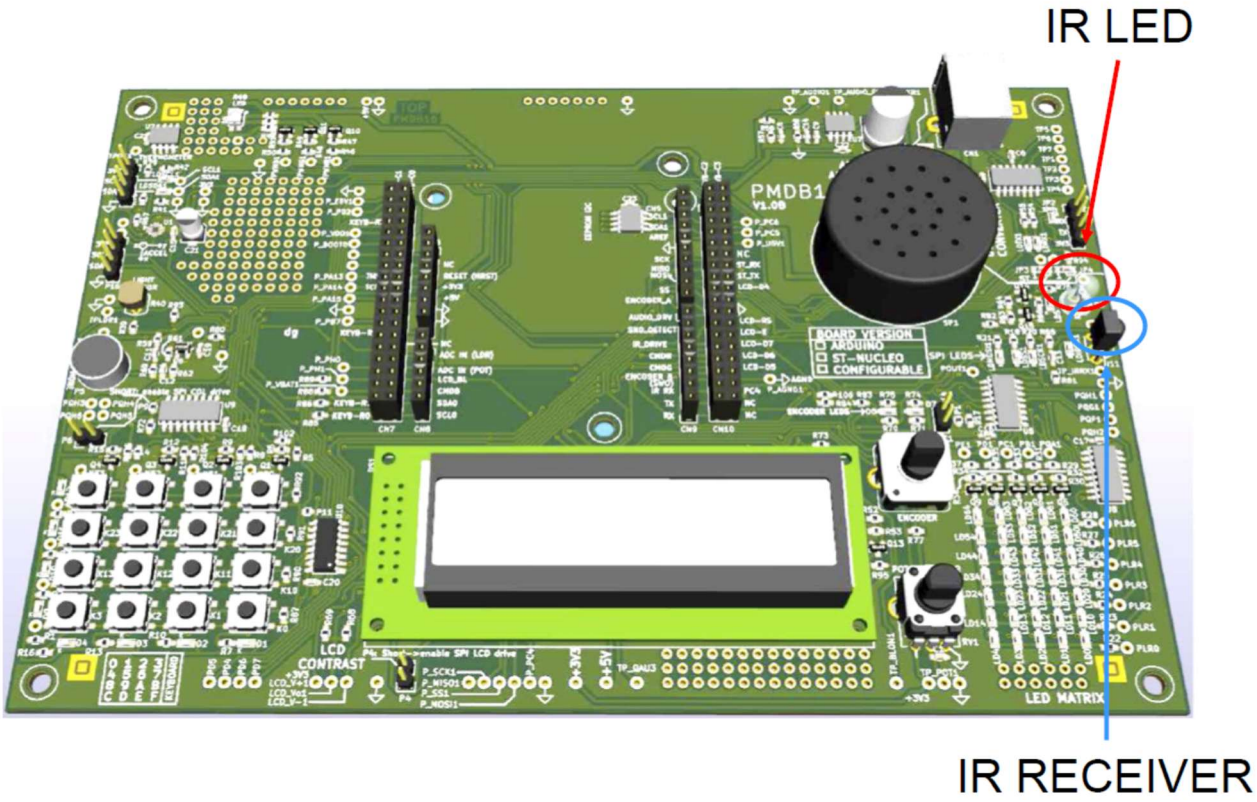
Lab12 – Comunicazione IR

Il lab consiste nel far parlare due schede PMDB16 senza fili. Si usa un **LED infrarosso come trasmettitore** e un **ricevitore IR integrato come ricevitore**, mettendoci sopra un **normale protocollo UART**.

1. Hardware sulla scheda

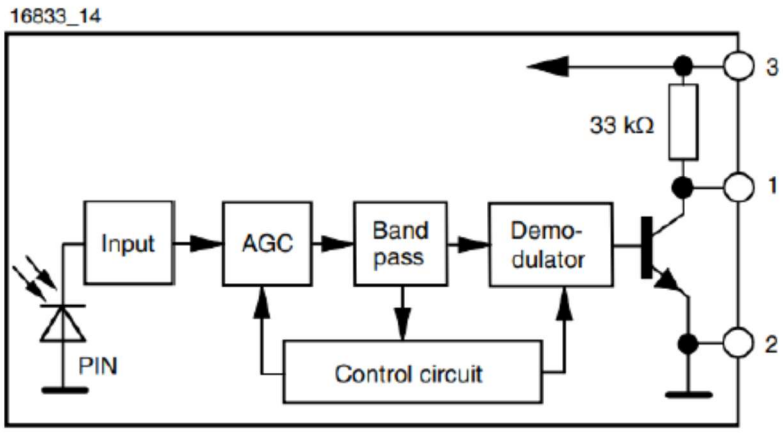
Il **LED IR (segnale IR_DRV)** è collegato a **PB10**, che è il canale 3 di TIM2 (**TIM2_CH3**). Serve quindi un **PWM hardware**. Il **ricevitore IR (segnale IR_IN)** è collegato a **PA10**, che è **USART1_RX**. Per il lab si usa la funzione USART1_RX, quindi il ricevitore diventa l'ingresso di una vera UART.



Ricevitore TSOP58238

Al suo interno ci sono, in sequenza:

- **un fotodiodo PIN;**
- **un amplificatore con AGC (Automatic Gain Control);**
- **un filtro passa-banda centrato a 38 kHz, seguito da un demodulatore;**
- **un'uscita open-collector con pull-up interna da 33 kΩ.**



Il punto chiave è che il ricevitore risponde solo a luce **modulata a 38 kHz**. La luce ambiente, continua o lenta, viene scartata dal passa-banda. Questo è il motivo per cui non basta accendere il LED in continua: bisogna pilotarlo con una portante a 38 kHz.

L'uscita è a **logica invertita**. È **alta** a riposo e va **bassa** solo mentre riceve la portante. Il diagramma "Output Function" delle slide mostra proprio questo: burst di 600 µs in ingresso corrispondono a impulsi bassi in uscita.

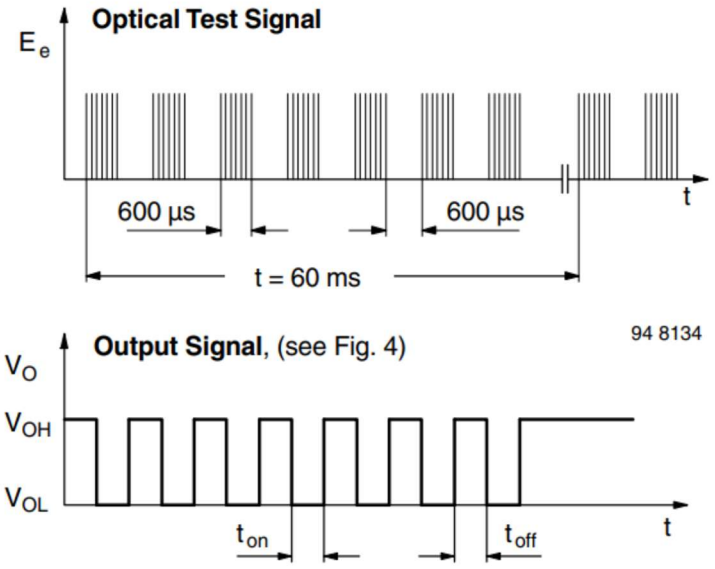
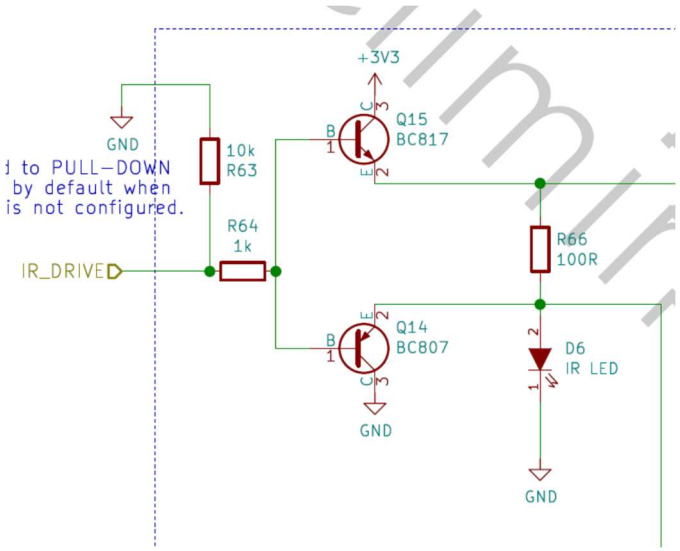


Fig. 3 - Output Function

Trasmettitore

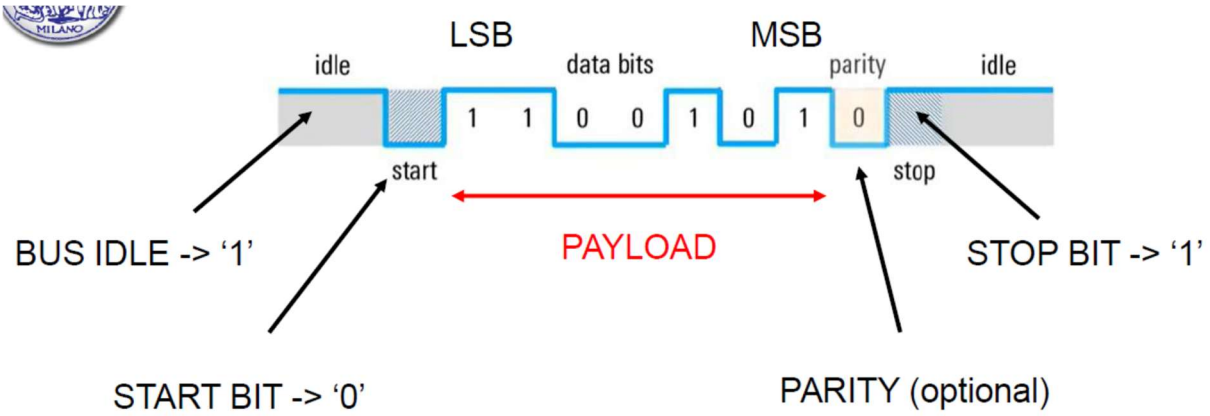
Il LED è un semplice **LED IR**. Un pin del micro eroga al massimo circa 8 mA, mentre al LED servono circa 20 mA. Per questo tra il pin e il LED c'è uno stadio di potenza **push-pull in classe B**: un BC817 (NPN) e un BC807 (PNP), con R64 da 1 kΩ in base e R66 da 100 Ω in serie al LED. La R63 da 10 kΩ tiene IR_DRIVE a massa di default, così il LED resta spento se il pin non è configurato. Le slide indicano di pilotarlo con **PWM a 38 kHz e duty cycle 50%**.



2. Teoria: UART su infrarosso

Una UART ha questo frame:

- linea a riposo (idle) a **1**;
- **start bit** a 0;
- **8 bit di dati**, prima l'LSB e per ultimo l'MSB;
- parità opzionale;
- **stop bit** a 1;
- tempo di bit = 1 / baud rate.



Bit logico	Trasmettitore	Uscita del ricevitore
1	LED spento (nessuna portante)	alta
0	LED che pulsa a 38 kHz	bassa

L'inversione del TSOP (portante = uscita bassa) coincide con la logica UART: idle = 1, start = 0. Perciò il pin PA10 può essere collegato **direttamente** al ricevitore USART1 senza inversioni software. Non è una coincidenza, è il motivo per cui questo schema funziona.

Perché baud rate ≤ 2400

A 2400 bps un bit dura circa 417 µs, cioè circa 16 periodi di portante a 38 kHz. Il TSOP ha bisogno di un certo numero di periodi consecutivi per riconoscere un burst (nel datasheet sono circa 10, ma verifica il valore esatto) e del tempo per l'AGC. **Con baud rate più alti i burst diventano troppo corti e il ricevitore non li vede o li distorce.**

3. Progetto 1: preliminare (gruppo diviso in TX e RX)

Scheda che trasmette

Step 1: inviare un byte a 2400 bps.

1. Configura un timer generico (TIMx) con interrupt ogni 1/2400 s. Con un clock timer di 84 MHz, per esempio PSC=99 e ARR=349, perché 84 MHz / 100 = 840 kHz e 840000 / 350 = 2400 Hz. È un calcolo mio, da verificare sul tuo clock tree.
2. Configura TIM2_CH3 in PWM a 38 kHz al 50% (a 84 MHz: PSC=0, ARR≈2210, CCR≈1105). Non avviare ancora il PWM.
3. Quando la funzione di invio viene chiamata, abilita TIMx in interrupt mode.
4. Nella callback del timer si fa una piccola **macchina a stati**, un bit per tick:
    - **start bit**: PWM acceso;
    - **8 bit dati** dal LSB: PWM acceso per un 0, spento per un 1;
    - **stop bit**: PWM spento, poi si ferma TIMx.

Step 2: inviare stringhe. Una funzione che chiama la funzione del byte per ogni carattere. Dato che l'invio è asincrono (interrupt), bisogna aspettare la fine del byte prima di passare al successivo, per esempio con un flag busy.

Scheda che riceve

Step 1: ricevere via USART2 in interrupt mode. USART2 è la seriale verso il PC. Si riceve 1 byte con HAL_UART_Receive_IT, e nella callback si riavvia subito la ricezione salvando il byte appena arrivato. Il byte ricevuto comanda il LED verde: **'1' lo accende, '0' lo spegne**. Per il debug si può scrivere in PuTTY e la scheda reagisce.

Step 2: passare all'IR. Si riceve da **USART1** a 2400 bps, sempre in interrupt mode, e il byte ricevuto viene inoltrato al PC su USART2. Le slide raccomandano di aiutare l'altra metà del gruppo a fare il debug del suo codice di invio, perché la ricezione è lo strumento per vedere se il TX funziona.

4. Progetto 2: comunicazione tra schede

Scheda TX: scansiona la matrice di pulsanti (come nel Lab 13, progetto 1b) e invia via IR UART il codice del tasto premuto.

Scheda RX: riceve il dato via IR UART e mostra il carattere corrispondente sulla matrice LED (come nel Lab 11, progetto 1b).

In alternativa: qualsiasi cosa a piacere, purché ci sia comunicazione IR.

5. Progetto 3: TX e RX sulla stessa scheda

Si integra tutto in una sola scheda, che sia trasmettitrice e ricevitrice. Serve gestire insieme TIM2_CH3 per il PWM, il timer del baud rate, USART1 in RX e i vari flag, senza conflitti tra le callback.

6. Note pratiche tratte dal tuo file "cose che sbaglio"

Alcuni errori già visti nei laboratori precedenti capitano di sicuro anche qui.

- **Callback condivisa dei timer.** Con due timer (uno per il baud rate, magari un altro per la scansione tastiera), HAL_TIM_PeriodElapsedCallback va scritta **una sola volta** e distinta con if (htim->Instance == TIMx). Stessa cosa per HAL_UART_RxCpltCallback, con USART1 e USART2 distinte tramite huart->Instance.
- **NVIC.** Se un timer o una USART in interrupt "non scatta mai", controlla in CubeMX la tab **NVIC Settings**. Vanno abilitati "TIMx global interrupt", "USART1 global interrupt" e "USART2 global interrupt". Anche per l'ARR: controlla di averlo calcolato e di non aver lasciato il default.
- **Ricezione a 1 byte.** Il buffer passato a HAL_UART_Receive_IT va passato con & se è una variabile scalare (&rx_byte), e senza & se è un array.
- **Ordine delle operazioni.** Tutto ciò che dipende dal byte ricevuto va nella callback di completamento, non subito dopo la chiamata a _IT, che è non bloccante.
- **Invio verso il PC.** HAL_UART_Transmit_IT o _DMA vogliono un buffer già pronto, non una variabile numerica per valore.
- **Riavvio della ricezione.** Se nella callback dimentichi di richiamare HAL_UART_Receive_IT, ricevi un solo byte e poi il silenzio.

7. Consigli di debug

Sono indicazioni mie, non dalle slide.

- Prima di collegare due schede, prova il TX con un oscilloscopio o un analizzatore logico su PB10 (portante a 38 kHz durante gli 0) e sull'uscita del ricevitore (impulsi bassi).
- Puoi controllare a occhio il LED IR con la fotocamera di un telefono, che vede l'infrarosso: se il PWM funziona, il LED appare viola/bianco.
- Tieni le due schede vicine (pochi cm) e ben allineate, almeno all'inizio.
- Le impostazioni della UART su entrambi i lati devono essere identiche: 2400 baud, 8 bit, nessuna parità, 1 stop bit.

Se vuoi, posso trasformare questa spiegazione in un documento con la teoria e le note pratiche del Lab 12 da aggiungere al progetto, accanto alle altre dispense.

Lab12_1Tx

Lab12_1Tx:

- Step 1: crea una funzione che invia un byte via Infrared UART a baud rate 2400 bps.
- Step 2: crea una funzione che invia una stringa via Infrared UART a baud rate 2400 bps

Task	Periferiche
Trasmissione IR	• Led IR (PB10) • Timer 2 (PWM 38kHz) • Timer 3 (Interrupt 2400 Hz)
PC Serial IN	• USART 2 (Baud Rate = 115200)

File .ioc

Pin

Funzione	Pin	Configurazione
LED IR (Tx)	PB10	TIM2_CH3
PC Serial IN	• PA2 • PA3	• USART2_TX • USART2_RX

Timers

TIMx	Ruolo	Modalità	Frequenza	Priority
TIM2	Modulare la luce IR (Tx)	PWM	38 kHz	/
TIM3	Bit Rate (Tx)	Global Interrupt	2400 Hz	0

- **Timer 2 (modulare la luce IR Tx):**
  - Channel 3: PWM GENERATION CH3
  - PSC=220
  - ARR=9
  - Pulse=5

$$f_{PWM} = \frac{f_{Timer}}{(ARR+1)(PSC+1)} = \frac{84\ MHz}{(9+1)\cdot(220+1)} = 38.01\ kHz$$

$$DC = \frac{CCR_x}{ARR+1} = \frac{5}{9+1} = 50\%$$

- **Timer 3 (bit rate):**
  - Clock Source: Internal Clock
  - TIM3 global interrupt: Enabled
  - PSC=349
  - ARR=99

$$f_{Timer} = \frac{f_{Clock}}{(ARR+1)(PSC+1)} = \frac{84\ MHz}{(99+1)(349+1)} = 2400\ Hz$$

Connectivity

- USART2:
  - Mode: Asynchronous
  - Baud Rate: 750
  - Word Length: 8
  - Parity: None

- Stop Bits: 1
- NVIC: USART2 global interrupt = Enabled

Codice

Private TypeDef

```
// Stati Tx IR
typedef enum {
    TX_STATE_IDLE  = 0,
    TX_STATE_START = 1,
    TX_STATE_DATA  = 2,
    TX_STATE_STOP  = 3
} TxState_t;
```

Private Variables

```
// UART2 RX (PC)
uint8_t rx_pc_buffer;
char received_pc_char;

// TX IR
volatile uint8_t  tx_ir_buffer;
volatile uint8_t  bit_counter = 0;
volatile TxState_t tx_state = TX_STATE_IDLE;
```

Funzioni

```
// Avvia l'invio di un singolo byte tramite IR UART
void IR_UART_SendByte(uint8_t data)
{
    // Solo se il canale TX è libero posso iniziare nuova trasmissione(IDLE)
    if (tx_state == TX_STATE_IDLE)
    {
        tx_ir_buffer = data;
        tx_state = TX_STATE_START;      // Ci prepariamo a inviare lo Start Bit

        __HAL_TIM_SET_COUNTER(&htim3, 0); // Azzera CNT di TIM3 affinchè lo Start Bit
duri un bit time pieno
        // e non di meno perchè magari CNT era ad un altro valore. CNT arriva a ARR
(99) prima di andare in
        // overflow e scattare l'ISR, se dalla scorsa ricezione CNT era ad un altro
valo-re non conta fino a
        // 100 ma di meno
        HAL_TIM_Base_Start_IT(&htim3); // TIM3 (2400 Hz) scandisce i bit nella Trasmis-
sione IR
    }
}

void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim)
{
    if (htim->Instance == TIM3) // 2400 Hz, scandisce i bit da mandare (TX IR)
    {
        switch (tx_state)
        {
            case TX_STATE_IDLE:
             // HIGH -> LED OFF
                HAL_TIM_PWM_Stop(&htim2, TIM_CHANNEL_3);
                break;

            case TX_STATE_START:
```

```
            // START BIT = LOW -> LED ON per 1/2400 s
                HAL_TIM_PWM_Start(&htim2, TIM_CHANNEL_3); // IR LED ON
                tx_state = TX_STATE_DATA; // Passa alla gestione dati
                bit_counter = 0;          // Resetta contatore bit dati
                break;

            case TX_STATE_DATA:
            // Inviamo 8 bit
            // Bit Masking
            // Esempio '1': 0(1)00 0001  >> 6  =  0000 000(1)
            // 0000 000(1) & 0000 0001 = 0000 0001
            // Esempio '1': (0)100 0001  >> 7  =  0000 000(0)
                        // 0000 000(0) & 0000 0001 = 0000 0000

                if (((tx_ir_buffer >> bit_counter) & 0x01) == 0)
                {
                    // 0 -> LED ON
                   HAL_TIM_PWM_Start(&htim2, TIM_CHANNEL_3);
                }
                else
                {
                    // 1 -> LED OFF
                   HAL_TIM_PWM_Stop(&htim2, TIM_CHANNEL_3);
                }

                bit_counter++; // Avanziamo nel buffer

                if (bit_counter >= 8) // Quando abbiamo finito
                {
                    tx_state = TX_STATE_STOP;
                }
                break;

            case TX_STATE_STOP:
            // STOP BIT = HIGH -> LED OFF
                HAL_TIM_PWM_Stop(&htim2, TIM_CHANNEL_3);

                // Ritorniamo in IDLE
                tx_state = TX_STATE_IDLE;
                // Finchè non premiamo altri tasti non abbiamo bisogno della Callback a
2400Hz
                HAL_TIM_Base_Stop_IT(&htim3);
                break;
        }
    }

}

void HAL_UART_RxCpltCallback(UART_HandleTypeDef *huart)
{
    if (huart->Instance == USART2){ // RICEVIAMO DAL PC
      received_pc_char=(char)rx_pc_buffer;

      // Siamo pronti a trasmettere IR
      IR_UART_SendByte(received_pc_char);

      // Ci rimettiamo in ascolto
      HAL_UART_Receive_IT(&huart2, &rx_pc_buffer, 1);
    }

}
```

Main

```
  // LCD
  lcd_initialize();

  lcd_backlight_ON();
  // Ricevere dal PC
  HAL_UART_Receive_IT(&huart2, &rx_pc_buffer, 1);
```

Lab12_1Rx

Lab12_1Rx:

- Step 1: ricevi i dati via UART, usando la USART2 in interrupt mode. Ad ogni byte ricevuto devi accendere LD2 se ricevi “1” e spegnerlo se ricevi “0”.
- Step 2: ora ricevi da IR USART1, a baud rate di 2400 bps. Manda la stringa via USART2 ad un remote terminal.

Task	Periferiche
Ricezione IR	• USART1 (Baud Rate=2400)
LCD	• Pin LCD

File .ioc

Pin

Funzione	**Pin**	**Configurazione**
IR (Rx)	PA10	USART1_RX
LCD	• PA4 • PB1 • PB2 • PB12 • PB13 • PB14 • PB15	• GPIO_Output • GPIO_Output • GPIO_Output • GPIO_Output • GPIO_Output • GPIO_Output • GPIO_Output

Connectivity

- **USART1:**
  - Mode: Asynchronous
  - Baud Rate: 2400
  - Word Length: 8
  - Parity: None
  - Stop Bits: 1
  - NVIC: USART1 global interrupt = Enabled

Codice

Private Define

```
#define LCD_SIZE 16
#define MAX_LEN  (2*LCD_SIZE)   // 32 caratteri: due righe
```

Private Variables

```
// UART1 RX (IR)
uint8_t rx_ir_buffer; // Byte ricevuto dall'IR
volatile uint8_t received_ir = 0;
char received_ir_char;


// LCD
char lcd_buffer_top[LCD_SIZE+1];
char lcd_buffer_bottom[LCD_SIZE+1];

int i=0;
int j=0;
int clear=0;
```

Funzioni

```
void HAL_UART_RxCpltCallback(UART_HandleTypeDef *huart)
{

    if (huart->Instance == USART1) // RICEVIAMO IR
    {
      received_ir_char=(char)rx_ir_buffer;

      if(received_ir_char=='\r'){ // Invio -> spazio
            received_ir_char=' ';
      }

      if(received_ir_char!='\n'){ // '\n' lo ignoro, altrimenti "\r\n" darebbe due
spazi

            if(clear){ // Le due righe erano piene: ricomincio
                  clear=0;
                  memset(lcd_buffer_top, ' ', LCD_SIZE);
                  memset(lcd_buffer_bottom, ' ', LCD_SIZE);
            }

            // Scriviamo nel buffer, un char alla volta
            // i caratteri arrivano a 13,3 ms l'uno dall'altro (a 750 baud) dal PC
alla scheda
            // e a 417 microsecondi l'uno dall'altro (a 2400 baud) da IR Tx a Rx
            // Siccome il buffer è cumulativo (lo riscriviamo)
```

```
            // e LCD è lento, vediamo direttamente la parola, anche se trasmettiamo un
byte alla volta
            if(j==0){
                  lcd_buffer_top[i]=received_ir_char;
            }else{
                  lcd_buffer_bottom[i]=received_ir_char;
            }

            // Spostiamo il cursore
            i++;
            if(i==LCD_SIZE){
                  i=0;
                  j=(j+1)%2;

                  if(j==0){
                        clear=1;
                  }
            }

            // Pronti a scrivere sull'LCD
            received_ir=1;
      }
      // Ci rimettiamo in ascolto
      HAL_UART_Receive_IT(&huart1, &rx_ir_buffer, 1);
   }

}
```

Main

```
  // LCD
  lcd_initialize();

  lcd_backlight_ON();

  // Ricevere da IR
  HAL_UART_Receive_IT(&huart1, &rx_ir_buffer, 1);
```

While(1)

```
      if (received_ir)
      {
        received_ir=0;
        lcd_println(lcd_buffer_top, 0);
        lcd_println(lcd_buffer_bottom, 1);
      }
```

Lab12_1TxRx (IR_STRINGA)

Task	Periferiche
Trasmissione IR	• Led IR (PB10)

	• Timer 2 (PWM 38kHz) • Timer 3 (Interrupt 2400 Hz)
Ricezione IR	• USART1 (Baud Rate=2400)
PC Serial IN	• USART 2 (Baud Rate = 115200)
LCD	• Pin LCD

File .ioc

Pin

Funzione	Pin	Configurazione
LED IR (Tx)	PB10	TIM2_CH3
IR (Rx)	PA10	USART1_RX
PC Serial IN	• PA2 • PA3	• USART2_TX • USART2_RX
LCD	• PA4 • PB1 • PB2 • PB12 • PB13 • PB14 • PB15	• GPIO_Output • GPIO_Output • GPIO_Output • GPIO_Output • GPIO_Output • GPIO_Output • GPIO_Output

Timers

TIMx	Ruolo	Modalità	Frequenza	Priority
TIM2	Modulare la luce IR (Tx)	PWM	38 kHz	/
TIM3	Bit Rate (Tx)	Global Interrupt	2400 Hz	0

- **Timer 2 (modulare la luce IR Tx)**:
  - Channel 3: PWM GENERATION CH3
  - PSC=220
  - ARR=9
  - Pulse=5

$$f_{PWM} = \frac{f_{Timer}}{(ARR+1)(PSC+1)} = \frac{84\ MHz}{(9+1)\cdot(220+1)} = 38.01\ kHz$$

$$DC = \frac{CCR_x}{ARR+1} = \frac{5}{9+1} = 50\%$$

- **Timer 3 (bit rate)**:
  - Clock Source: Internal Clock
  - TIM3 global interrupt: Enabled
  - PSC=349
  - ARR=99

$$f_{Timer} = \frac{f_{Clock}}{(ARR+1)(PSC+1)} = \frac{84\ MHz}{(99+1)(349+1)} = 2400\ Hz$$

Connectivity

- **USART1**:
  - Mode: Asynchronous
  - Baud Rate: 2400
  - Word Length: 8
  - Parity: None
  - Stop Bits: 1
  - NVIC: USART1 global interrupt = Enabled

- USART2:
  - Mode: Asynchronous
  - Baud Rate: 750
  - Word Length: 8
  - Parity: None
  - Stop Bits: 1
  - NVIC: USART2 global interrupt = Enabled

Codice

Private TypeDef

```
// Stati Tx IR
typedef enum {
    TX_STATE_IDLE  = 0,
    TX_STATE_START = 1,
    TX_STATE_DATA  = 2,
    TX_STATE_STOP  = 3
} TxState_t;
```

Private Define

```
#define LCD_SIZE 16
#define MAX_LEN  (2*LCD_SIZE)   // 32 caratteri: due righe
```

Private Variables

```
// UART2 RX (PC)
uint8_t rx_pc_buffer;
```

```
char received_pc_char;

// TX IR
volatile uint8_t  tx_ir_buffer;
volatile uint8_t  bit_counter = 0;
volatile TxState_t tx_state = TX_STATE_IDLE;

// UART1 RX (IR)
uint8_t rx_ir_buffer; // Byte ricevuto dall'IR
volatile uint8_t received_ir = 0;
char received_ir_char;


// LCD
char lcd_buffer_top[LCD_SIZE+1];
char lcd_buffer_bottom[LCD_SIZE+1];

int i=0;
int j=0;
int clear=0;
```

Funzioni

```
// Avvia l'invio di un singolo byte tramite IR UART
void IR_UART_SendByte(uint8_t data)
{
    // Solo se il canale TX è libero posso iniziare nuova trasmissione(IDLE)
    if (tx_state == TX_STATE_IDLE)
    {
        tx_ir_buffer = data;
        tx_state = TX_STATE_START;      // Ci prepariamo a inviare lo Start Bit

        __HAL_TIM_SET_COUNTER(&htim3, 0); // Azzera CNT di TIM3 affinchè lo Start Bit
duri un bit time pieno
        // e non di meno perchè magari CNT era ad un altro valore. CNT arriva a ARR
(99) prima di andare in
        // overflow e scattare l'ISR, se dalla scorsa ricezione CNT era ad un altro
valo-re non conta fino a
        // 100 ma di meno
        HAL_TIM_Base_Start_IT(&htim3); // TIM3 (2400 Hz) scandisce i bit nella Trasmis-
sione IR
    }
}

void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim)
{
    if (htim->Instance == TIM3) // 2400 Hz, scandisce i bit da mandare (TX IR)
    {
        switch (tx_state)
        {
            case TX_STATE_IDLE:
             // HIGH -> LED OFF
                HAL_TIM_PWM_Stop(&htim2, TIM_CHANNEL_3);
                break;

            case TX_STATE_START:
             // START BIT = LOW -> LED ON per 1/2400 s
                HAL_TIM_PWM_Start(&htim2, TIM_CHANNEL_3); // IR LED ON
                tx_state = TX_STATE_DATA; // Passa alla gestione dati
                bit_counter = 0;          // Resetta contatore bit dati
                break;
```

```
            case TX_STATE_DATA:
             // Inviamo 8 bit
             // Bit Masking
             // Esempio '1': 0(1)00 0001  >> 6  =  0000 000(1)
             // 0000 000(1) & 0000 0001 = 0000 0001
             // Esempio '1': (0)100 0001  >> 7  =  0000 000(0)
                          // 0000 000(0) & 0000 0001 = 0000 0000

                if (((tx_ir_buffer >> bit_counter) & 0x01) == 0)
                {
                    // 0 -> LED ON
                   HAL_TIM_PWM_Start(&htim2, TIM_CHANNEL_3);
                }
                else
                {
                    // 1 -> LED OFF
                   HAL_TIM_PWM_Stop(&htim2, TIM_CHANNEL_3);
                }

                bit_counter++; // Avanziamo nel buffer

                if (bit_counter >= 8) // Quando abbiamo finito
                {
                    tx_state = TX_STATE_STOP;
                }
                break;

            case TX_STATE_STOP:
             // STOP BIT = HIGH -> LED OFF
                HAL_TIM_PWM_Stop(&htim2, TIM_CHANNEL_3);

                // Ritorniamo in IDLE
                tx_state = TX_STATE_IDLE;
                // Finchè non premiamo altri tasti non abbiamo bisogno della Callback a
2400Hz
                HAL_TIM_Base_Stop_IT(&htim3);
                break;
        }
    }

}

void HAL_UART_RxCpltCallback(UART_HandleTypeDef *huart)
{

    if (huart->Instance == USART1) // RICEVIAMO IR
    {
      received_ir_char=(char)rx_ir_buffer;

      if(received_ir_char=='\r'){ // Invio -> spazio
            received_ir_char=' ';
      }

      if(received_ir_char!='\n'){ // '\n' lo ignoro, altrimenti "\r\n" darebbe due
spazi

             if(clear){ // Le due righe erano piene: ricomincio
                   clear=0;
                   memset(lcd_buffer_top, ' ', LCD_SIZE);
                   memset(lcd_buffer_bottom, ' ', LCD_SIZE);
```

```
            }

            // Scriviamo nel buffer, un char alla volta
            // i caratteri arrivano a 13,3 ms l'uno dall'altro (a 750 baud) dal PC
alla scheda
            // e a 417 microsecondi l'uno dall'altro (a 2400 baud) da IR Tx a Rx
            // Siccome il buffer è cumulativo (lo riscriviamo)
            // e LCD è lento, vediamo direttamente la parola, anche se trasmettiamo un
byte alla volta
            if(j==0){
                  lcd_buffer_top[i]=received_ir_char;
            }else{
                  lcd_buffer_bottom[i]=received_ir_char;
            }

            // Spostiamo il cursore
            i++;
            if(i==LCD_SIZE){
                  i=0;
                  j=(j+1)%2;

                  if(j==0){
                        clear=1;
                  }
            }

            // Pronti a scrivere sull'LCD
            received_ir=1;
      }
      // Ci rimettiamo in ascolto
      HAL_UART_Receive_IT(&huart1, &rx_ir_buffer, 1);
   }

   if (huart->Instance == USART2){ // RICEVIAMO DAL PC
      received_pc_char=(char)rx_pc_buffer;

      // Siamo pronti a trasmettere IR
      IR_UART_SendByte(received_pc_char);

      // Ci rimettiamo in ascolto
      HAL_UART_Receive_IT(&huart2, &rx_pc_buffer, 1);
   }

}
```

Main

```
  // LCD
  lcd_initialize();

  lcd_backlight_ON();
  // Ricevere dal PC
  HAL_UART_Receive_IT(&huart2, &rx_pc_buffer, 1);
  // Ricevere da IR
  HAL_UART_Receive_IT(&huart1, &rx_ir_buffer, 1);
```

While(1)

```
      if (received_ir)
```

```
{
  received_ir=0;
  lcd_println(lcd_buffer_top, 0);
  lcd_println(lcd_buffer_bottom, 1);
}
```

Lab12_2Tx

Lab12_2Tx: scannerizza la keyboard e manda il dato corrispondente via IR UART.

Task	Periferiche
Trasmissione IR	• Led IR (PB10) • Timer 2 (PWM 38kHz) • Timer 3 (Interrupt 2400 Hz)
Scan Keyboard	• Timer 4 (Interrupt 500 Hz)

File .ioc

Pin

Funzione	Pin	Configurazione
LED IR (Tx)	PB10	TIM2_CH3
Riga Keyboard	PC2	GPIO_Input
Riga Keyboard	PC3	GPIO_Input
Riga Keyboard	PC12	GPIO_Input
Riga Keyboard	PC13	GPIO_Input
Colonna Keyboard	PC8	GPIO_Output
Colonna Keyboard	PC9	GPIO_Output
Colonna Keyboard	PC10	GPIO_Output
Colonna Keyboard	PC11	GPIO_Output

Timers

TIMx	Ruolo	Modalità	Frequenza	Priority
TIM2	Modulare la luce IR (Tx)	PWM	38 kHz	/
TIM3	Bit Rate (Tx)	Global Interrupt	2400 Hz	0
TIM4	Scansionare la Keyboard	Global Interrupt	500 Hz	1

- **Timer 2 (modulare la luce IR Tx):**

- Channel 3: PWM GENERATION CH3
- PSC=220
- ARR=9
- Pulse=5

$$f_{PWM} = \frac{f_{Timer}}{(ARR+1)(PSC+1)} = \frac{84\ MHz}{(9+1)\cdot(220+1)} = 38.01\ kHz$$

$$DC = \frac{CCR_x}{ARR+1} = \frac{5}{9+1} = 50\%$$

- **Timer 3 (bit rate)**:
  - Clock Source: Internal Clock
  - TIM3 global interrupt: Enabled
  - PSC=349
  - ARR=99

$$f_{Timer} = \frac{f_{Clock}}{(ARR+1)(PSC+1)} = \frac{84\ MHz}{(99+1)(349+1)} = 2400\ Hz$$

- **Timer 4 (scansionare la tastiera)**:
  - Clock Source: Internal Clock
  - TIM4 global interrupt: Enabled
  - PSC=8399
  - ARR=19

$$f_{Timer} = \frac{f_{Clock}}{(ARR+1)(PSC+1)} = \frac{84\ MHz}{(19+1)(8399+1)} = 500\ Hz$$

Codice

Private TypeDef

```
// Stati Tx IR
typedef enum {
    TX_STATE_IDLE  = 0,
    TX_STATE_START = 1,
    TX_STATE_DATA  = 2,
    TX_STATE_STOP  = 3
} TxState_t;

// Keyboard
typedef struct {
      char character;
      uint8_t duration;
      uint8_t sent;
      uint8_t idx;
} button;

typedef struct {
    GPIO_TypeDef* port;
    uint16_t pin;
```

```
} PinMap;
```

Private Define

```
// KEYBOARD
// Keyboard Rows/Columns
#define KEY_C0 GPIOC,GPIO_PIN_11
#define KEY_C1 GPIOC,GPIO_PIN_10
#define KEY_C2 GPIOC,GPIO_PIN_9
#define KEY_C3 GPIOC,GPIO_PIN_8
#define KEY_R0 GPIOC,GPIO_PIN_3
#define KEY_R1 GPIOC,GPIO_PIN_2
#define KEY_R2 GPIOC,GPIO_PIN_13
#define KEY_R3 GPIOC,GPIO_PIN_12

// Keyboard Dimension
#define DIM 4

// Keyboard Buttons
#define DEBOUNCE 3
#define MAX_DURATION 250
```

Private Variables

```
// KEYBOARD
PinMap columns[] = {
    { KEY_C3 },
    { KEY_C2 },
    { KEY_C1 },
    { KEY_C0 }
};

PinMap rows[] = {
    { KEY_R0 },
    { KEY_R1 },
    { KEY_R2 },
    { KEY_R3 }
};


volatile button keyboard[DIM][DIM] = {
    { {'C', 0, 0, 12}, {'D', 0, 0, 13}, {'E', 0, 0, 14}, {'F', 0, 0, 15} },
    { {'8', 0, 0, 8}, {'9', 0, 0, 9}, {'A', 0, 0, 10}, {'B', 0, 0, 11} },
    { {'4', 0, 0, 4}, {'5', 0, 0, 5}, {'6', 0, 0, 6}, {'7', 0, 0, 7} },
    { {'0', 0, 0, 0}, {'1', 0, 0, 1}, {'2', 0, 0, 2}, {'3', 0, 0, 3} }
};

uint8_t active_col = 0;



// TX IR
volatile uint8_t  tx_data_buffer = 'G';
volatile uint8_t  bit_counter = 0;
volatile TxState_t tx_state = TX_STATE_IDLE;
```

Funzioni

```
// Avvia l'invio di un singolo byte tramite IR UART
void IR_UART_SendByte(uint8_t data)
```

```
{
    // Solo se il canale TX è libero posso iniziare nuova trasmissione(IDLE)
    if (tx_state == TX_STATE_IDLE)
    {
        tx_data_buffer = data;
        tx_state = TX_STATE_START;      // Ci prepariamo a inviare lo Start Bit

        __HAL_TIM_SET_COUNTER(&htim3, 0); // Azzera CNT di TIM3 affinchè lo Start Bit
duri un bit time pieno
        // e non di meno perchè magari CNT era ad un altro valore. CNT arriva a ARR
(99) prima di andare in
        // overflow e scattare l'ISR, se dalla scorsa ricezione CNT era ad un altro va-
lore non conta fino a
        // 100 ma di meno
        HAL_TIM_Base_Start_IT(&htim3); // TIM3 (2400 Hz) scandisce i bit nella Trasmis-
sione IR
    }
}


void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim)
{
    if (htim->Instance == TIM3) // 2400 Hz, scandisce i bit da mandare (TX IR)
    {
        switch (tx_state)
        {
            case TX_STATE_IDLE:
             // HIGH -> LED OFF
                HAL_TIM_PWM_Stop(&htim2, TIM_CHANNEL_3);
                break;

            case TX_STATE_START:
             // START BIT = LOW -> LED ON per 1/2400 s
                HAL_TIM_PWM_Start(&htim2, TIM_CHANNEL_3); // IR LED ON
                tx_state = TX_STATE_DATA; // Passa alla gestione dati
                bit_counter = 0;          // Resetta contatore bit dati
                break;

            case TX_STATE_DATA:
             // Inviamo 8 bit
             // Bit Masking
             // Esempio '1': 0(1)00 0001  >> 6  =  0000 000(1)
             // 0000 000(1) & 0000 0001 = 0000 0001
             // Esempio '1': (0)100 0001  >> 7  =  0000 000(0)
                          // 0000 000(0) & 0000 0001 = 0000 0000

                if (((tx_data_buffer >> bit_counter) & 0x01) == 0)
                {
                    // 0 -> LED ON
                   HAL_TIM_PWM_Start(&htim2, TIM_CHANNEL_3);
                }
                else
                {
                    // 1 -> LED OFF
                   HAL_TIM_PWM_Stop(&htim2, TIM_CHANNEL_3);
                }

                bit_counter++; // Avanziamo nel buffer

                if (bit_counter >= 8) // Quando abbiamo finito
                {
```

```
                    tx_state = TX_STATE_STOP;
                }
                break;

            case TX_STATE_STOP:
             // STOP BIT = HIGH -> LED OFF
                HAL_TIM_PWM_Stop(&htim2, TIM_CHANNEL_3);

                // Ritorniamo in IDLE
                tx_state = TX_STATE_IDLE;
                // Finchè non premiamo altri tasti non abbiamo bisogno della Callback a
2400Hz
                HAL_TIM_Base_Stop_IT(&htim3);
                break;
        }
    }

    if (htim->Instance == TIM4) { // 500 Hz (scansione colonne KEYBOARD)
              for(int r=0; r<DIM; r++){
                   // Se il tasto è premuto
                  if(HAL_GPIO_ReadPin(rows[r].port, rows[r].pin) == GPIO_PIN_RESET){
                   // Se è premuto da poco
                      if(keyboard[r][active_col].duration < MAX_DURATION ) {
                          keyboard[r][active_col].duration++; // Aggiorniamo
                      }
                  } else { // Se il tasto non è premuto azzeriamo tutto
                      keyboard[r][active_col].duration = 0;
                      keyboard[r][active_col].sent = 0;
                  }
              }
              // Deselezioniamo la colonna appena letta
              HAL_GPIO_WritePin(columns[active_col].port, columns[active_col].pin,
GPIO_PIN_RESET);

              // Aggiorniamo la colonna da attivare
              active_col=(active_col+1) % DIM;

              // Attiviamo la nuova colonna
              HAL_GPIO_WritePin(columns[active_col].port, columns[active_col].pin,
GPIO_PIN_SET);
       }

}
```

Main

```
  // Prepara la scansione della keyboard
  HAL_TIM_Base_Start_IT(&htim4);
  HAL_GPIO_WritePin(columns[active_col].port, columns[active_col].pin, GPIO_PIN_SET);
```

While(1)

```
        // --- KEYBOARD SCANNING ---
              for(int j=0;j<DIM;j++){
                          for(int i=0;i<DIM;i++){
                                if((keyboard[i][j].duration>=DEBOUNCE)&&(key-
board[i][j].sent==0) ){
```

```
                                        IR_UART_SendByte(keyboard[i][j].character);
                                        keyboard[i][j].sent=1;
                                    }
                                }
                            }
```

Lab12_2Rx

Lab12_2Rx: ricevi via IR UART il dato e trasmetti il simbolo corrispondente sulla matrice di LED.

Task	Periferiche
Ricezione IR	• USART1 (Baud Rate=2400)
Matrice Led	• Pin Matrice Led • Timer 5 (Interrupt 250Hz) • SPI1

File .ioc

Pin

Funzione	Pin	Configurazione
IR (Rx)	PA10	USART1_RX
SCK	PA5	SPI1
MISO	PA6	SPI1
MOSI	PA7	SPI1
RCLK	PB6	GPIO_Output

Timers

TIMx	Ruolo	Modalità	Frequenza	Priority
TIM5	Refreshare colonne Led	Global Interrupt	250 Hz	0

- **Timer 5 (refreshare le colonne della matrice LED)**:
  - Clock Source: Internal Clock
  - TIM3 global interrupt: Enabled (priority = 0)
  - PSC=99
  - ARR=3359

$$f_{Timer} = \frac{f_{Clock}}{(ARR+1)(PSC+1)} = \frac{84\ MHz}{(3359+1)(99+1)} = 250\ Hz$$

Connectivity

- **USART1:**
  - Mode: Asynchronous
  - Baud Rate: 2400
  - Word Length: 8
  - Parity: None
  - Stop Bits: 1
  - NVIC: USART1 global interrupt = Enabled

- SPI1:
  - Mode: Full Duplex Master
  - Data size: 8 bits
  - Prescaler (for Baud Rate): 4
  - DMA:
    - Request: TX (Memory to Peripheral)
    - Priority: High
    - Mode: Normal
  - NVIC:
    - DMA2 stream3 global interrupt = Enabled
    - SPI1 global interrupt = Enabled

Codice

Private TypeDef

```
// Keyboard
typedef struct {
	char character;
	uint8_t idx;
} button;
```

Private Define

```
// Keyboard Dimension
#define DIM 4
// LED MATRIX
#define NUM_COLS_LED 5
#define NUM_SYMBOLS 16
#define RCLK_PORT GPIOB
#define RCLK_PIN  GPIO_PIN_6
```

Private Variables

```
volatile button keyboard[DIM][DIM] = {
    { {'C', 12}, {'D', 13}, {'E', 14}, {'F', 15} },
    { {'8', 8}, {'9', 9}, {'A', 10}, {'B', 11} },
    { {'4', 4}, {'5', 5}, {'6', 6}, {'7', 7} },
    { {'0', 0}, {'1', 1}, {'2', 2}, {'3', 3} }
```

```
};



// RX IR
uint8_t rx_data_buffer; // Byte ricevuto dall'IR
volatile uint8_t ir_received_char = 0;
volatile uint8_t ir_data_ready = 0;

// LED MATRIX
static const uint8_t COL_LED[NUM_COLS_LED] = {16, 8, 4, 2, 1};
static const uint8_t SYMBOLS[NUM_SYMBOLS+1][NUM_COLS_LED] = {
    { 62,  69,  73,  81,  62}, // '0'
    {  0,  33, 127,   1,   0}, // '1'
    { 33,  67,  69,  73,  49}, // '2'
    { 34,  65,  73,  73,  54}, // '3'
    { 12,  20,  36, 127,   4}, // '4'
    {114,  81,  81,  81,  78}, // '5'
    { 30,  41,  73,  73,   6}, // '6'
    { 64,  71,  72,  80,  96}, // '7'
    { 54,  73,  73,  73,  54}, // '8'
    { 48,  73,  73,  74,  60}, // '9'
    { 63,  72,  72,  72,  63}, // 'A'
    {127,  73,  73,  73,  54}, // 'B'
    { 62,  65,  65,  65,  34}, // 'C'
    {127,  65,  65,  65,  62}, // 'D'
    {127,  73,  73,  73,  65}, // 'E'
    {127,  72,  72,  72,  64}, // 'F'
    {  0,   0,   0,   0,   0}  // spento
};
static volatile uint8_t sym_idx = NUM_SYMBOLS;   // cosa mostra la matrice (spenta se
NUM_SYMBOLS)
static uint8_t col_cnt = 0;               // colonna della matrice in refresh
```

Funzioni

```
static inline void spi_send(uint8_t col, uint8_t row){
      static uint8_t buffer[2]; // 2 byte
      // DAISY RING
      buffer[0]=row; // prima la riga
      buffer[1]=col; // poi la colonna

      // Ogni 4 ms trasmettiamo (refreshamo)
      HAL_SPI_Transmit_DMA(&hspi1, buffer, 2);
}



void HAL_SPI_TxCpltCallback(SPI_HandleTypeDef *hspi){
      if(hspi->Instance==SPI1){
            // Una volta terminata la comunicazione dei 2 byte
            // diamo un impulso su RCLK per far scattare il latch
            // dei due shift register
            HAL_GPIO_WritePin(RCLK_PORT, RCLK_PIN, GPIO_PIN_SET);
            HAL_GPIO_WritePin(RCLK_PORT, RCLK_PIN, GPIO_PIN_RESET);
      }
}



void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim)
```

```
{
    if (htim->Instance == TIM5) { // 4ms, refresh colonne
      spi_send(COL_LED[col_cnt], SYMBOLS[sym_idx][col_cnt]);
      col_cnt=(col_cnt+1)%NUM_COLS_LED;
    }
}

void HAL_UART_RxCpltCallback(UART_HandleTypeDef *huart)
{
    if (huart->Instance == USART1)
    {
      ir_received_char = rx_data_buffer;
      ir_data_ready = 1; // Siamo pronti a scrivere sulla matrice Led
       HAL_UART_Receive_IT(&huart1, &rx_data_buffer, 1); // ci rimettiamo in ascolto
    }

}
```

Main

```
  // Prepara alla ricezione la UART1
  HAL_UART_Receive_IT(&huart1, &rx_data_buffer, 1);


  // Matrice Led
  HAL_TIM_Base_Start_IT(&htim5);
```

While(1)

```
              // MATRICE LED
              if (ir_data_ready==1){
                    // Cerca l'idx del simbolo da mostrare

                    // Tramite i for
                    for (int j=0; j<DIM; j++){
                          for (int i=0; i<DIM; i++){
                                if(ir_received_char == keyboard[i][j].character){
                                      sym_idx=keyboard[i][j].idx;
                                }
                          }
                    }
                    ir_data_ready = 0;

                    /* o tramite la logica ASCII
                     * if (ir_data_ready) {
                               sym_idx = char_to_idx(ir_received_char);
                               ir_data_ready = 0;
                         }
                     */
              }
```

Lab12_3

Lab12_3: scannerizza la keyboard e manda il dato corrispondente via IR UART, ricevi sulla stessa board via IR UART il dato e trasmetti il simbolo corrispondente sulla matrice di LED.

Task	Periferiche
Trasmissione IR	• Led IR (PB10) • Timer 2 (PWM 38kHz) • Timer 3 (Interrupt 2400 Hz)
Ricezione IR	• USART1 (Baud Rate=2400)
Scan Keyboard	• Timer 4 (Interrupt 500 Hz)
Matrice Led	• Pin Matrice Led • Timer 5 (Interrupt 250Hz) • SPI1

File .ioc

Pin

Funzione	Pin	Configurazione
LED IR (Tx)	PB10	TIM2_CH3
IR (Rx)	PA10	USART1_RX
Riga Keyboard	PC2	GPIO_Input
Riga Keyboard	PC3	GPIO_Input
Riga Keyboard	PC12	GPIO_Input
Riga Keyboard	PC13	GPIO_Input
Colonna Keyboard	PC8	GPIO_Output
Colonna Keyboard	PC9	GPIO_Output
Colonna Keyboard	PC10	GPIO_Output
Colonna Keyboard	PC11	GPIO_Output
SCK	PA5	SPI1
MISO	PA6	SPI1
MOSI	PA7	SPI1
RCLK	PB6	GPIO_Output

Timers

TIMx	Ruolo	Modalità	Frequenza	Priority
TIM2	Modulare la luce IR (Tx)	PWM	38 kHz	/

TIM3	Bit Rate (Tx)	Global Interrupt	2400 Hz	0
TIM4	Scansionare la Keyboard	Global Interrupt	500 Hz	1
TIM5	Refreshare colonne Led	Global Interrupt	250 Hz	2

- **Timer 2 (modulare la luce IR Tx):**
  - Channel 3: PWM GENERATION CH3
  - PSC=220
  - ARR=9
  - Pulse=5

$$f_{PWM} = \frac{f_{Timer}}{(ARR+1)(PSC+1)} = \frac{84\ MHz}{(9+1)\cdot(220+1)} = 38.01\ kHz$$

$$DC = \frac{CCR_x}{ARR+1} = \frac{5}{9+1} = 50\%$$

- **Timer 3 (bit rate):**
  - Clock Source: Internal Clock
  - TIM3 global interrupt: Enabled
  - PSC=349
  - ARR=99

$$f_{Timer} = \frac{f_{Clock}}{(ARR+1)(PSC+1)} = \frac{84\ MHz}{(99+1)(349+1)} = 2400\ Hz$$

- **Timer 4 (scansionare la tastiera):**
  - Clock Source: Internal Clock
  - TIM4 global interrupt: Enabled
  - PSC=8399
  - ARR=19

$$f_{Timer} = \frac{f_{Clock}}{(ARR+1)(PSC+1)} = \frac{84\ MHz}{(19+1)(8399+1)} = 500\ Hz$$

- **Timer 5 (refreshare le colonne della matrice LED)**:
  - Clock Source: Internal Clock
  - TIM3 global interrupt: Enabled (priority = 2)
  - PSC=99
  - ARR=3359

$$f_{Timer} = \frac{f_{Clock}}{(ARR+1)(PSC+1)} = \frac{84\ MHz}{(3359+1)(99+1)} = 250\ Hz$$

Connectivity

- **USART1:**
  - Mode: Asynchronous
  - Baud Rate: 2400
  - Word Length: 8
  - Parity: None
  - Stop Bits: 1
  - NVIC: USART1 global interrupt = Enabled

- SPI1:
  - Mode: Full Duplex Master
  - Data size: 8 bits
  - Prescaler (for Baud Rate): 4
  - DMA:
    - Request: TX (Memory to Peripheral)
    - Priority: High
    - Mode: Normal
  - NVIC:
    - DMA2 stream3 global interrupt = Enabled
    - SPI1 global interrupt = Enabled

Codice

Private TypeDef

```
// Stati Tx IR
typedef enum {
    TX_STATE_IDLE  = 0,
    TX_STATE_START = 1,
    TX_STATE_DATA  = 2,
    TX_STATE_STOP  = 3
} TxState_t;

// Keyboard
typedef struct {
       char character;
       uint8_t duration;
       uint8_t sent;
       uint8_t idx;
} button;

typedef struct {
    GPIO_TypeDef* port;
    uint16_t pin;
} PinMap;
```

Private Define

```
// KEYBOARD
// Keyboard Rows/Columns
#define KEY_C0 GPIOC,GPIO_PIN_11
#define KEY_C1 GPIOC,GPIO_PIN_10
#define KEY_C2 GPIOC,GPIO_PIN_9
#define KEY_C3 GPIOC,GPIO_PIN_8
#define KEY_R0 GPIOC,GPIO_PIN_3
#define KEY_R1 GPIOC,GPIO_PIN_2
#define KEY_R2 GPIOC,GPIO_PIN_13
#define KEY_R3 GPIOC,GPIO_PIN_12

// Keyboard Dimension
#define DIM 4

// Keyboard Buttons
#define DEBOUNCE 3
#define MAX_DURATION 250

// LED MATRIX
#define NUM_COLS_LED 5
#define NUM_SYMBOLS 16
#define RCLK_PORT GPIOB
#define RCLK_PIN  GPIO_PIN_6
```

Private Variables

```
// KEYBOARD
PinMap columns[] = {
    { KEY_C3 },
    { KEY_C2 },
    { KEY_C1 },
    { KEY_C0 }
};

PinMap rows[] = {
    { KEY_R0 },
    { KEY_R1 },
    { KEY_R2 },
    { KEY_R3 }
};


volatile button keyboard[DIM][DIM] = {
    { {'C', 0, 0, 12}, {'D', 0, 0, 13}, {'E', 0, 0, 14}, {'F', 0, 0, 15} },
    { {'8', 0, 0, 8}, {'9', 0, 0, 9}, {'A', 0, 0, 10}, {'B', 0, 0, 11} },
    { {'4', 0, 0, 4}, {'5', 0, 0, 5}, {'6', 0, 0, 6}, {'7', 0, 0, 7} },
    { {'0', 0, 0, 0}, {'1', 0, 0, 1}, {'2', 0, 0, 2}, {'3', 0, 0, 3} }
};

uint8_t active_col = 0;



// TX IR
volatile uint8_t  tx_data_buffer = 'G';
volatile uint8_t  bit_counter = 0;
volatile TxState_t tx_state = TX_STATE_IDLE;

// RX IR
uint8_t rx_data_buffer; // Byte ricevuto dall'IR
```

```
volatile uint8_t ir_received_char = 0;
volatile uint8_t ir_data_ready = 0;

// LED MATRIX
static const uint8_t COL_LED[NUM_COLS_LED] = {16, 8, 4, 2, 1};
static const uint8_t SYMBOLS[NUM_SYMBOLS+1][NUM_COLS_LED] = {
    { 62,  69,  73,  81,  62}, // '0'
    {  0,  33, 127,   1,   0}, // '1'
    { 33,  67,  69,  73,  49}, // '2'
    { 34,  65,  73,  73,  54}, // '3'
    { 12,  20,  36, 127,   4}, // '4'
    {114,  81,  81,  81,  78}, // '5'
    { 30,  41,  73,  73,   6}, // '6'
    { 64,  71,  72,  80,  96}, // '7'
    { 54,  73,  73,  73,  54}, // '8'
    { 48,  73,  73,  74,  60}, // '9'
    { 63,  72,  72,  72,  63}, // 'A'
    {127,  73,  73,  73,  54}, // 'B'
    { 62,  65,  65,  65,  34}, // 'C'
    {127,  65,  65,  65,  62}, // 'D'
    {127,  73,  73,  73,  65}, // 'E'
    {127,  72,  72,  72,  64}, // 'F'
    {  0,   0,   0,   0,   0}  // spento
};
static volatile uint8_t sym_idx = NUM_SYMBOLS;   // cosa mostra la matrice (spenta se
NUM_SYMBOLS)
static uint8_t col_cnt = 0;                // colonna della matrice in refresh
```

Funzioni

```
static inline void spi_send(uint8_t col, uint8_t row){
      static uint8_t buffer[2]; // 2 byte
      // DAISY RING
      buffer[0]=row; // prima la riga
      buffer[1]=col; // poi la colonna

      // Ogni 4 ms trasmettiamo (refreshamo)
      HAL_SPI_Transmit_DMA(&hspi1, buffer, 2);
}

// Avvia l'invio di un singolo byte tramite IR UART
void IR_UART_SendByte(uint8_t data)
{
    // Solo se il canale TX è libero posso iniziare nuova trasmissione(IDLE)
    if (tx_state == TX_STATE_IDLE)
    {
        tx_data_buffer = data;
        tx_state = TX_STATE_START;      // Ci prepariamo a inviare lo Start Bit

        __HAL_TIM_SET_COUNTER(&htim3, 0); // Azzera CNT di TIM3 affinchè lo Start Bit
duri un bit time pieno
        // e non di meno perchè magari CNT era ad un altro valore. CNT arriva a ARR
(99) prima di andare in
        // overflow e scattare l'ISR, se dalla scorsa ricezione CNT era ad un altro va-
lore non conta fino a
        // 100 ma di meno
        HAL_TIM_Base_Start_IT(&htim3); // TIM3 (2400 Hz) scandisce i bit nella Trasmis-
sione IR
    }
}
```

```c
void HAL_SPI_TxCpltCallback(SPI_HandleTypeDef *hspi){
      if(hspi->Instance==SPI1){
            // Una volta terminata la comunicazione dei 2 byte
            // diamo un impulso su RCLK per far scattare il latch
            // dei due shift register
            HAL_GPIO_WritePin(RCLK_PORT, RCLK_PIN, GPIO_PIN_SET);
            HAL_GPIO_WritePin(RCLK_PORT, RCLK_PIN, GPIO_PIN_RESET);
      }
}



void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim)
{
    if (htim->Instance == TIM3) // 2400 Hz, scandisce i bit da mandare (TX IR)
    {
        switch (tx_state)
        {
            case TX_STATE_IDLE:
            // HIGH -> LED OFF
                HAL_TIM_PWM_Stop(&htim2, TIM_CHANNEL_3);
                break;

            case TX_STATE_START:
            // START BIT = LOW -> LED ON per 1/2400 s
                HAL_TIM_PWM_Start(&htim2, TIM_CHANNEL_3); // IR LED ON
                tx_state = TX_STATE_DATA; // Passa alla gestione dati
                bit_counter = 0;          // Resetta contatore bit dati
                break;

            case TX_STATE_DATA:
            // Inviamo 8 bit
            // Bit Masking
            // Esempio '1': 0(1)00 0001  >> 6  =  0000 000(1)
            // 0000 000(1) & 0000 0001 = 0000 0001
            // Esempio '1': (0)100 0001  >> 7  =  0000 000(0)
                        // 0000 000(0) & 0000 0001 = 0000 0000

                if (((tx_data_buffer >> bit_counter) & 0x01) == 0)
                {
                    // 0 -> LED ON
                   HAL_TIM_PWM_Start(&htim2, TIM_CHANNEL_3);
                }
                else
                {
                    // 1 -> LED OFF
                   HAL_TIM_PWM_Stop(&htim2, TIM_CHANNEL_3);
                }

                bit_counter++; // Avanziamo nel buffer

                if (bit_counter >= 8) // Quando abbiamo finito
                {
                    tx_state = TX_STATE_STOP;
                }
                break;

            case TX_STATE_STOP:
            // STOP BIT = HIGH -> LED OFF
                HAL_TIM_PWM_Stop(&htim2, TIM_CHANNEL_3);
```

```
                // Ritorniamo in IDLE
                tx_state = TX_STATE_IDLE;
                // Finchè non premiamo altri tasti non abbiamo bisogno della Callback a
2400Hz
                HAL_TIM_Base_Stop_IT(&htim3);
                break;
        }
    }

    if (htim->Instance == TIM4) { // 500 Hz (scansione colonne KEYBOARD)
              for(int r=0; r<DIM; r++){
                   // Se il tasto è premuto
                  if(HAL_GPIO_ReadPin(rows[r].port, rows[r].pin) == GPIO_PIN_RESET){
                   // Se è premuto da poco
                      if(keyboard[r][active_col].duration < MAX_DURATION ) {
                          keyboard[r][active_col].duration++; // Aggiorniamo
                      }
                  } else { // Se il tasto non è premuto azzeriamo tutto
                      keyboard[r][active_col].duration = 0;
                      keyboard[r][active_col].sent = 0;
                  }
              }
              // Deselezioniamo la colonna appena letta
              HAL_GPIO_WritePin(columns[active_col].port, columns[active_col].pin,
GPIO_PIN_RESET);

              // Aggiorniamo la colonna da attivare
              active_col=(active_col+1) % DIM;

              // Attiviamo la nuova colonna
              HAL_GPIO_WritePin(columns[active_col].port, columns[active_col].pin,
GPIO_PIN_SET);
       }


    if (htim->Instance == TIM5) { // 4ms, refresh colonne
      spi_send(COL_LED[col_cnt], SYMBOLS[sym_idx][col_cnt]);
      col_cnt=(col_cnt+1)%NUM_COLS_LED;
    }



}

void HAL_UART_RxCpltCallback(UART_HandleTypeDef *huart)
{
    if (huart->Instance == USART1)
    {
      ir_received_char = rx_data_buffer;
      ir_data_ready = 1; // Siamo pronti a scrivere sulla matrice Led
        HAL_UART_Receive_IT(&huart1, &rx_data_buffer, 1); // ci rimettiamo in ascolto
    }

}
```

Main

```
  // Prepara alla ricezione la UART1
  HAL_UART_Receive_IT(&huart1, &rx_data_buffer, 1);
```

```
  // Prepara la scansione della keyboard
  HAL_TIM_Base_Start_IT(&htim4);
  HAL_GPIO_WritePin(columns[active_col].port, columns[active_col].pin, GPIO_PIN_SET);

  // Matrice Led
  HAL_TIM_Base_Start_IT(&htim5);
```

While(1)

```
        // --- KEYBOARD SCANNING ---
              for(int j=0;j<DIM;j++){
                          for(int i=0;i<DIM;i++){
                                if((keyboard[i][j].duration>=DEBOUNCE)&&(key-
board[i][j].sent==0) ){
                                      IR_UART_SendByte(keyboard[i][j].character);
                                      keyboard[i][j].sent=1;
                                }
                          }
              }

              // MATRICE LED
              if (ir_data_ready==1){
                    // Cerca l'idx del simbolo da mostrare

                    // Tramite i for
                    for (int j=0; j<DIM; j++){
                          for (int i=0; i<DIM; i++){
                                if(ir_received_char == keyboard[i][j].character){
                                      sym_idx=keyboard[i][j].idx;
                                }
                          }
                    }
                    ir_data_ready = 0;

                    /* o tramite la logica ASCII
                     * if (ir_data_ready) {
                               sym_idx = char_to_idx(ir_received_char);
                               ir_data_ready = 0;
                         }
                     */
              }
```